AI-POWERED BANKING CUSTOMER SUPPORT

Intelligence & Response Automation

FINAL PROJECT DOCUMENTATION

Purpose • Requirements • Applications • Architecture • Implementation • Evaluation • Code Reference • Testing • Submission Guide

Final official evaluation: DistilBERT Accuracy 88.67% | Weighted F1 88.16%

Traditional ML baseline: Accuracy 87.79% | Weighted F1 87.87%

Prepared as the final project guide/documentation package.

1. Document Purpose

This document consolidates the final project purpose, project requirements, business applications, dataset, workflow, implementation modules, model results, confidence and escalation policy, Streamlit application, testing procedure, limitations, future enhancements, and code reference into one submission-oriented PDF.

The project specification requires an end-to-end banking customer-support solution combining Dataset → NLP Processing → ML Baseline → Deep Learning → Intent Classification → Confidence & Escalation → Generative AI → Streamlit Application → optional Deployment. ■filecite■turn2file0■L53-L57■

The official project deliverables include dataset/data understanding, EDA, ML baseline, deep learning model, confidence/escalation, Generative AI, Streamlit business application, project documentation, and final presentation. ■filecite■turn2file0■L251-L314■

2. Problem Statement

Banks receive large volumes of customer-support queries involving card payments, withdrawals, transfers, verification, card delivery, fees and other services. Manual reading, categorization, prioritization and response handling increases support workload, response time and the possibility of inconsistent routing.

The objective is to build an AI-powered banking customer-support intelligence and response-automation system that automatically understands customer queries, classifies banking intent, estimates confidence and priority, generates controlled support responses, identifies uncertain cases for human escalation, summarizes support information and provides operational analytics. ■filecite■turn31file6■

3. Business Purpose and Applications

1. Automated Customer Query Classification — identify the intent behind incoming banking queries and route them to the appropriate support category.

2. Intelligent Customer Response Generation — generate relevant, professional and context-aware responses.

3. Confidence-Based Escalation — identify uncertain predictions and recommend human-agent intervention.

4. Conversation Intelligence — extract customer issue, intent, sentiment, priority, key information and recommended action.

5. Customer-Support Analytics — monitor frequent issues, intent distribution, confidence, escalation levels, sentiment and high-priority cases. ■filecite■turn31file6■

4. Official Requirements

Requirement	Final implementation/status
Dataset & data preparation	Official train.csv/test.csv; validation, cleaning and processed datasets
EDA	Category distribution, text length, quality and imbalance analysis
ML baseline	TF-IDF + Logistic Regression
Deep learning	Fine-tuned DistilBERT, 77 classes
Official evaluation	Reserved official test set; accuracy, precision, recall, F1, confusion matrix, confidence
Confidence & escalation	High / medium / low policy with automated, verification and human escalation
Risk & ambiguity	Refined ambiguity checks, close-prediction detection and semantic safety guard
Generative AI	Controlled response layer using model intent/confidence and safe response logic
Conversation intelligence	Support summary and recommended action information
Streamlit	Support Assistant + Model Analytics business UI
Documentation	README + this consolidated project documentation

Requirement	Final implementation/status
Deployment	Optional / N/A for the final local demonstration

5. Dataset

Official dataset name: Banking Customer Support Query Dataset. The specification defines train.csv and test.csv as the official project dataset, with text and category columns and 77 intent categories. ■filecite■turn31file1■

Property	train.csv	test.csv
Rows	10,003	3,080
Columns	2	2
Categories	77	77
Missing cells	0	0
Duplicate rows	0	0
Test distribution	—	40 examples/category

• Official test.csv is reserved for final evaluation and must not be used for training or tuning.

• Processed files: train_clean.csv, test_clean.csv and label_mapping.json.

• The final label mapping contains 77 official classes.

6. Data Preparation and EDA

Preprocessing validated columns, missing values, duplicate records, text normalization and label consistency. After cleaning, the processed training data contained 9,999 rows and the processed test data contained 3,079 rows.

EDA identified 77 categories. The most frequent training category had 187 examples and the least frequent had 35. Text length statistics: mean about 59.41 characters, median 47, Q1 36, Q3 64, maximum 429. The IQR was 28 and the upper IQR boundary was 106; long queries were treated as potentially legitimate rather than blindly removed.

7. End-to-End System Architecture

Customer Query → Text/NLP Processing → TF-IDF ML Baseline → DistilBERT Intent Classification → Confidence → Risk/Ambiguity Detection → Escalation Decision → Controlled AI Response → Conversation Summary → Streamlit Analytics

The Streamlit UI currently exposes a Support Assistant and Model Analytics workflow. The application sidebar documents the operational workflow as Customer Query → DistilBERT → Intent + Confidence → Risk & Ambiguity Detection → Escalation Decision → Safe AI Response. ■filecite■turn31file8■

8. Machine Learning Baseline

A traditional NLP baseline was created before transformer modeling so that the project could demonstrate measurable improvement over a conventional approach. The final baseline uses TF-IDF n-grams with Logistic Regression.

Configuration: TF-IDF n-grams 1–2, min_df=2, max_df=0.95, sublinear_tf=True, up to 50,000 features; Logistic Regression with max_iter=2000, C=4.0 and saga solver.

Official test result: 87.79% accuracy and 87.87% weighted F1.

9. Deep Learning — DistilBERT

The final transformer is DistilBERT fine-tuned for 77-class banking intent classification. The project uses a stratified training/validation split and selects the best validation checkpoint before final official test evaluation.

Experiment 2 settings: distilbert-base-uncased, batch size 16, learning rate 2e-5, max sequence length 64, weight decay 0.01, warmup ratio 0.10, checkpointing and best-model selection.

Validation progression: Epoch 1 accuracy 62.15%; Epoch 2 82.25%; Epoch 3 86.65%; best validation accuracy 88.95% with weighted F1 88.76%. Total runtime was approximately 160.62 minutes on CPU.

Final best model: dataset/checkpoints_step3_exp2/best_model. The model was verified as a 77-label DistilBERT model with approximately 67 million parameters.

10. Official Model Evaluation

Metric	ML Baseline	DistilBERT
Accuracy	87.79%	88.67%
Weighted Precision	88.45%	88.33%
Weighted Recall	87.79%	88.67%
Weighted F1	87.87%	88.16%
Macro F1	87.87%	88.16%

• Accuracy improvement over ML baseline: +0.88 percentage points.

• Weighted F1 improvement: +0.29 percentage points based on rounded official values.

• The official test set was used only for final evaluation; no further tuning should be performed against it.

11. Prediction Confidence

The final official test confidence profile was: mean 75.32%, median 83.27%, minimum 10.86%, maximum 95.86%. High confidence (≥80%) accounted for 1,756/3,079 predictions (57.03%); medium confidence (60–<80%) accounted for 686 (22.28%); low confidence (<60%) accounted for 637 (20.69%).

Band	Rule	Count	Percentage
High	≥ 80%	1,756	57.03%
Medium	60% to <80%	686	22.28%
Low	<60%	637	20.69%

12. Confidence, Risk and Escalation

• High confidence ≥80% → AUTOMATED_RESPONSE.

• Medium confidence 60–<80% → VERIFY_WITH_CUSTOMER.

• Low confidence <60% → HUMAN_ESCALATION.

• Refined risk detection adds ambiguity, close-top-prediction and semantic-safety checks.

• A semantic safety guard was added for cases such as “issue with debit card” being incorrectly mapped toward direct-debit intent.

Error analysis showed that confidence is useful but is not identical to correctness. Some high-confidence incorrect predictions remained, so the final application deliberately combines confidence with semantic/risk controls rather than treating confidence as proof of correctness.

13. Error Analysis

• Official test size: 3,079 processed records.

• Correct predictions: 2,730.

• Incorrect predictions: 349.

• Accuracy: 88.6652% (reported as 88.67%).

• Error rate: 11.3348%.

• Weakest identified class: virtual_card_not_working, with zero precision/recall/F1 in the final error-analysis report.

• Important confusion pairs included virtual_card_not_working → getting_virtual_card and why_verify_identity → verify_my_identity.

• Confidence bands showed low-confidence predictions were substantially less reliable than high-confidence predictions.

14. Generative AI and Conversation Intelligence

The final project includes a controlled response-generation layer. The application uses the predicted intent, confidence and risk decision to determine how the response should be handled.

Low-confidence cases are handled conservatively with clarification or escalation. Medium-confidence cases request verification. High-confidence cases can receive an intent-aware support response subject to semantic and safety controls.

Conversation intelligence is represented through support summaries that capture the customer issue, intent, confidence/risk context, priority and recommended action, matching the project requirement for issue, intent, sentiment, priority, key information and action. ■filecite■turn31file0■

15. Streamlit Business Application

The final Streamlit application is app.py. It provides a colorful business-style interface with two main tabs: Support Assistant and Model Analytics.

• Customer query input

• Predicted banking intent

• Prediction confidence

• Priority

• Risk and ambiguity decision

• Escalation recommendation

• Top-3 predictions

• Controlled AI response

• Safety information

• Support summary

• Technical details

• Model analytics and final evaluation metrics

The project specification explicitly requires the Streamlit application to provide query input, predicted category, confidence, priority, AI response, escalation recommendation, conversation summaries, recommended support action and analytics. ■filecite■turn31file1■

16. Project Structure

```
AI-Powered Banking Customer Support/
■■■ app.py
■■■ README.md
■■■ requirements.txt
■■■ STEP_4_FINAL_APPLICATION_TEST.txt
■■■ dataset/
■   ■■■ raw/
■   ■   ■■■ train.csv
■   ■   ■■■ test.csv
■   ■■■ processed/
■   ■   ■■■ train_clean.csv
■   ■   ■■■ test_clean.csv
■   ■   ■■■ label_mapping.json
■   ■■■ checkpoints_step3_exp2/
■       ■■■ best_model/
■■■ notebooks/
■■■ src/
■   ■■■ 01_data_preprocessing.py
■   ■■■ 02_machine_learning_baseline.py
■   ■■■ 03_deep_learning_bert.py
■   ■■■ 03b_deep_learning_exp2.py
■   ■■■ 04_evaluate_model.py
■   ■■■ 05_inference.py
■   ■■■ 06_confidence_escalation.py
■   ■■■ 07_generative_ai.py
■   ■■■ 08_error_analysis.py
■   ■■■ 08b_risk_detection.py
■   ■■■ 09_project_requirement_audit.py
■■■ results/
    ■■■ ml/
    ■■■ evaluation/
    ■■■ step3_exp2/
    ■■■ error_analysis/
    ■■■ confidence/
    ■■■ generative_ai/
    ■■■ reports/
```

17. Code Reference — Final Source Files

File	Purpose
01_data_preprocessing.py	Validate raw data, clean text, create processed datasets and label mapping.
02_machine_learning_baseline.py	TF-IDF feature extraction and Logistic Regression baseline.
03_deep_learning_bert.py	Earlier/alternative transformer implementation retained for project history.
03b_deep_learning_exp2.py	Final full DistilBERT fine-tuning experiment and checkpointing.
04_evaluate_model.py	Official test-set evaluation and classification metrics/confusion analysis.
05_inference.py	Single-query inference using the final model and official mapping.
06_confidence_escalation.py	Confidence thresholds and operational escalation decisions.
07_generative_ai.py	Controlled support-response generation layer.
08_error_analysis.py	Misclassification, confusion and confidence-vs-correctness analysis.
08b_risk_detection.py	Refined ambiguity and semantic safety detection.
09_project_requirement_audit.py	Automated project requirement audit.
app.py	Final Streamlit business application integrating the complete workflow.

18. Core Commands / How to Run

```
# Activate environment
.\.venv\Scripts\Activate.ps1

# Final requirement audit
python src\09_project_requirement_audit.py

# Start Streamlit
streamlit run app.py

# Local URL
http://localhost:8501
```

19. Final Application Test

The separate Step 4 test file contains five final queries, expected behaviors, analytics checks and a submission checklist. The test procedure is included below for convenience.

```
============================================================
```

STEP 4 — FINAL APPLICATION TEST
AI-POWERED BANKING CUSTOMER SUPPORT
==

PURPOSE

Run the final Streamlit application and verify the main
customer-support features before project submission.

IMPORTANT

- Do NOT retrain the model.
- Do NOT modify the official test dataset.
- Do NOT change the final best_model.
- This is only a final application verification.
- Keep the original project and previous files unchanged.

----
1. OPEN THE FINAL PROJECT FOLDER

Project folder:

C:\Users\Gokul\OneDrive\Desktop\AI-Powered Banking Customer Support\FINAL_PROJECT\AI-Powered Banking Customer Support

----
2. ACTIVATE THE VIRTUAL ENVIRONMENT

Run:

.\.venv\Scripts\Activate.ps1

You should see:

(.venv)

at the beginning of the PowerShell prompt.

----
3. START THE STREAMLIT APPLICATION

Run:

streamlit run app.py

The browser should open the Streamlit application.

If the browser does not open automatically, copy the
Local URL shown in PowerShell and open it in your browser.

Example:

http://localhost:8501

----
4. TEST QUERY #1 — HIGH CONFIDENCE

Enter:

I don't recognize a direct debit payment from my account

Check that the application displays:

[ ] Predicted intent
[ ] Confidence score
[ ] Priority
[ ] Risk decision
[ ] Escalation decision
[ ] AI response
[ ] Top-3 predictions
[ ] Support summary
[ ] Safety information

Expected general behavior:

- High confidence
- Automated response
- No unnecessary escalation
- Controlled banking response

Take a screenshot for project evidence.

----
5. TEST QUERY #2 — LOW CONFIDENCE

Enter:

account locked

Check that the application displays:

[ ] Low confidence
[ ] Human escalation
[ ] Conservative/safe response
[ ] Risk information
[ ] Support summary

Expected general behavior:

- Low confidence
- HUMAN_ESCALATION
- Safe response instead of blindly answering

Take a screenshot for project evidence.

---

6. TEST QUERY #3 — AMBIGUOUS / SAFETY CASE

Enter:

issue with debit card

Check that the application displays:

[ ] Intent prediction
[ ] Confidence
[ ] Risk detection
[ ] Verification/escalation decision
[ ] Safe response

Expected general behavior:

- Risk/safety detection should be triggered
- The system should avoid blindly treating the query as
  an unrelated direct-debit problem
- Verification should be requested when appropriate

Take a screenshot for project evidence.

---

7. TEST QUERY #4 — SHORT QUERY

Enter:

card damaged

Check that the application displays:

[ ] Confidence
[ ] Risk flags
[ ] Escalation decision
[ ] Safe response

Expected general behavior:

- Low/uncertain confidence may occur
- Human escalation is acceptable
- Conservative response is expected

Take a screenshot for project evidence.

---

8. TEST QUERY #5 — NORMAL BANKING QUERY

Enter:

How can I transfer money to another bank account?

Check that the application displays:

[ ] Predicted intent
[ ] Confidence
[ ] Priority
[ ] AI response
[ ] Escalation decision
[ ] Top-3 predictions
[ ] Support summary

Take a screenshot for project evidence.

```
9. CHECK MODEL ANALYTICS
------------------------------------------------------------

Open the:

Model Analytics

tab.

Verify that the application displays:

DistilBERT Accuracy:
88.67%

DistilBERT Weighted F1:
88.16%

ML Baseline Accuracy:
87.79%

ML Baseline Weighted F1:
87.87%

Accuracy Improvement:
+0.88 percentage points

Also verify the confidence distribution and
error-analysis information.

Take a screenshot of the analytics page.


------------------------------------------------------------
10. FINAL APPLICATION CHECKLIST
------------------------------------------------------------

[ ] Streamlit application starts successfully
[ ] Main page loads without errors
[ ] Support Assistant tab works
[ ] Model Analytics tab works
[ ] Query input works
[ ] Intent prediction appears
[ ] Confidence appears
[ ] Priority appears
[ ] Risk detection appears
[ ] Escalation decision appears
[ ] AI response appears
[ ] Top-3 predictions appear
[ ] Support summary appears
[ ] Safety information appears
[ ] Analytics metrics appear
[ ] No Python error is displayed
[ ] No model-loading error is displayed


------------------------------------------------------------
11. AFTER TESTING
------------------------------------------------------------

If all important checks pass:

FINAL APPLICATION STATUS: READY FOR SUBMISSION

Do NOT retrain the model after this test.

Do NOT modify the official test dataset.

Keep screenshots as project evidence.

Recommended screenshot evidence:

01_main_application.png
02_high_confidence.png
03_low_confidence_escalation.png
04_safety_verification.png
05_short_query_escalation.png
06_normal_banking_query.png
07_model_analytics.png


------------------------------------------------------------
12. IF AN ERROR OCCURS
------------------------------------------------------------

DO NOT change multiple files at once.

Copy the complete error message from PowerShell/browser
and provide it to the project guide.

We will diagnose the exact cause first and then make the
smallest required fix.

============================================================
```

```
END OF STEP 4 — FINAL APPLICATION TEST
============================================================
```

20. Available Final Code Appendix

The following final source files were available in the conversation/project workspace and are reproduced in full in this appendix: app.py and the corrected 09_project_requirement_audit.py. The remaining source files are part of the final project folder and are listed in Section 17; this PDF does not invent or reconstruct source code that was not available to the document generator.

20A. app.py — Complete Available Source

```
# ============================================================
# AI-POWERED BANKING CUSTOMER SUPPORT
# FINAL STREAMLIT APPLICATION - STEP 8C
# ============================================================
# Uses:
#   - Fine-tuned DistilBERT
#   - Official 77-class label mapping
#   - Confidence analysis
#   - Refined risk & ambiguity detection
#   - Confidence-aware escalation
#   - Safe customer-support response
#   - Final official evaluation analytics
#
# IMPORTANT:
# No model training or test-set evaluation is performed here.
# ============================================================

import os
import sys
import json
import html
import inspect
import importlib
import streamlit as st


# ============================================================
# 1. PROJECT PATHS
# ============================================================

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
SRC_DIR = os.path.join(BASE_DIR, "src")
RESULTS_DIR = os.path.join(BASE_DIR, "results")
EVALUATION_DIR = os.path.join(RESULTS_DIR, "evaluation")

if SRC_DIR not in sys.path:
    sys.path.insert(0, SRC_DIR)


# ============================================================
# 2. PAGE CONFIGURATION
# ============================================================

st.set_page_config(
    page_title="AI Banking Customer Support",
    page_icon="■",
    layout="wide",
    initial_sidebar_state="expanded",
)


# ============================================================
# 3. CUSTOM CSS
# ============================================================

st.markdown(
    """
    <style>
    .stApp {
        background: linear-gradient(
            135deg,
            #eef4ff 0%,
            #f7f4ff 45%,
            #eefcf8 100%
        );
    }

    .main .block-container {
        max-width: 1450px;
        padding-top: 1.5rem;
        padding-bottom: 3rem;
        padding-left: 2rem;
        padding-right: 2rem;
    }

    .hero {
        background: linear-gradient(
            135deg,
            #0f172a 0%,
            #1d4ed8 45%,
            #7c3aed 100%
        );
        border-radius: 26px;
        padding: 2.2rem 2.5rem;
        margin-bottom: 1.5rem;

        color: white;
        box-shadow: 0 15px 40px rgba(37, 99, 235, 0.25);
    }
```

```
.hero-title {
    font-size: 2.45rem;
    font-weight: 850;
    letter-spacing: -0.8px;
    margin-bottom: 0.4rem;
}

.hero-subtitle {
    font-size: 1.05rem;
    line-height: 1.65;
    opacity: 0.92;
    max-width: 950px;
}

.section-title {
    color: #172554;
    font-size: 1.35rem;
    font-weight: 800;
    margin-top: 1.2rem;
    margin-bottom: 0.8rem;
}

.info-card {
    background: white;
    border-radius: 19px;
    padding: 1.25rem;
    min-height: 135px;
    border: 1px solid #dbeafe;
    box-shadow: 0 7px 24px rgba(15, 23, 42, 0.08);
}

.card-icon {
    font-size: 1.65rem;
    margin-bottom: 0.35rem;
}

.card-label {
    color: #64748b;
    font-size: 0.78rem;
    font-weight: 750;
    text-transform: uppercase;
    letter-spacing: 0.7px;
}

.card-value {
    color: #172554;
    font-size: 1.18rem;
    font-weight: 850;
    margin-top: 0.4rem;
    word-break: break-word;
}

.query-card {
    background: linear-gradient(135deg, #ffffff, #f7fbff);
    border-radius: 19px;
    padding: 1.3rem 1.5rem;
    border: 1px solid #bfdbfe;
    box-shadow: 0 6px 22px rgba(30, 64, 175, 0.08);
    font-size: 1.05rem;
    line-height: 1.65;
}

.response-card {
    background: white;
    border-radius: 20px;
    border-left: 7px solid #2563eb;
    padding: 1.5rem 1.6rem;
    box-shadow: 0 8px 28px rgba(15, 23, 42, 0.09);
    color: #1e293b;
    font-size: 1.04rem;
    line-height: 1.8;
}

.risk-card {
    background: #fff7ed;
    border: 1px solid #fed7aa;
    border-left: 7px solid #f97316;
    border-radius: 18px;
    padding: 1.2rem 1.4rem;
    line-height: 1.7;
}

.safe-card {
    background: #f0fdf4;
    border: 1px solid #bbf7d0;
    border-left: 7px solid #22c55e;
    border-radius: 18px;
    padding: 1.2rem 1.4rem;
    line-height: 1.7;
}

.badge-high, .badge-medium, .badge-low,
.priority-normal, .priority-review, .priority-high {
```

```
        display: inline-block;
        padding: 0.45rem 0.9rem;
        border-radius: 999px;
        font-weight: 800;
    }

    .badge-high, .priority-normal {
        background: #dcfce7;
        color: #166534;
    }

    .badge-medium, .priority-review {
        background: #fef3c7;
        color: #92400e;
    }

    .badge-low, .priority-high {
        background: #fee2e2;
        color: #991b1b;
    }

    section[data-testid="stSidebar"] {
        background: linear-gradient(
            180deg,
            #0f172a 0%,
            #172554 45%,
            #312e81 100%
        );
    }

    section[data-testid="stSidebar"] * {
        color: white !important;
    }

    .stButton > button {
        min-height: 3.2rem;
        border-radius: 15px;
        border: none;
        font-weight: 800;
        font-size: 1rem;
        background: linear-gradient(135deg, #2563eb, #7c3aed);
        color: white;
        box-shadow: 0 8px 20px rgba(79, 70, 229, 0.28);
    }

    .stButton > button:hover {
        background: linear-gradient(135deg, #1d4ed8, #6d28d9);
        color: white;
    }

    .footer {
        text-align: center;
        color: #64748b;
        font-size: 0.85rem;
        padding-top: 1.5rem;
    }
    </style>
    """,
    unsafe_allow_html=True,
)


# ============================================================
# 4. LOAD AI PIPELINES
# ============================================================

@st.cache_resource
def load_ai_pipeline():
    return importlib.import_module("07_generative_ai")


@st.cache_resource
def load_risk_pipeline():
    return importlib.import_module("08b_risk_detection")


try:
    ai = load_ai_pipeline()
except Exception as e:
    st.error("Unable to load the AI response pipeline.")
    st.exception(e)
    st.stop()


try:
    risk = load_risk_pipeline()
except Exception as e:
    risk = None


# ============================================================
# 5. HELPERS
```

```
# ============================================================

def get_priority(confidence_level):
    if confidence_level == "HIGH":
        return "Normal"
    if confidence_level == "MEDIUM":
        return "Review"
    return "High Attention"


def get_priority_badge(priority):
    if priority == "Normal":
        return '<span class="priority-normal">■ Normal</span>'
    if priority == "Review":
        return '<span class="priority-review">■ Review</span>'
    return '<span class="priority-high">■ High Attention</span>'


def get_confidence_badge(level):
    if level == "HIGH":
        return '<span class="badge-high">■ HIGH</span>'
    if level == "MEDIUM":
        return '<span class="badge-medium">■ MEDIUM</span>'
    return '<span class="badge-low">■ LOW</span>'


def load_final_metrics():
    path = os.path.join(EVALUATION_DIR, "final_metrics.json")
    if not os.path.exists(path):
        return None
    try:
        with open(path, "r", encoding="utf-8") as f:
            return json.load(f)
    except Exception:
        return None


def confidence_from_result(result):
    """Return top-1 confidence as 0..1."""
    if "confidence" in result:
        try:
            return float(result["confidence"])
        except Exception:
            pass

    if "confidence_percent" in result:
        try:
            return float(result["confidence_percent"]) / 100.0
        except Exception:
            pass

    return 0.0


def top2_confidence_from_result(result):
    top = result.get("top_predictions", [])
    if len(top) >= 2:
        try:
            return float(top[1].get("confidence", 0.0))
        except Exception:
            pass
    return 0.0


def fallback_refined_risk(query, predicted_category, top1, top2):
    """
    Local safety fallback matching the refined Step 8B policy.
    It is only used if the risk module does not expose a callable
    public assessment function.
    """
    q = query.lower().strip()
    flags = []
    reasons = []

    # Confidence bands
    if top1 < 0.60:
        flags.append("LOW_CONFIDENCE")
        reasons.append("Prediction confidence is below 60%.")
        decision = "HUMAN_ESCALATION"
        risk_level = "HIGH"
    elif top1 < 0.80:
        flags.append("MEDIUM_CONFIDENCE")
        reasons.append("Prediction confidence is between 60% and 80%.")
        decision = "VERIFY_WITH_CUSTOMER"
        risk_level = "MEDIUM"
    else:
        decision = "AUTOMATED_RESPONSE"
        risk_level = "LOW"

    # Close competing predictions
    margin = max(top1 - top2, 0.0)
    if top1 >= 0.80 and margin < 0.10:
```

```
        flags.append("CLOSE_TOP_PREDICTIONS")
        reasons.append("The top two intent probabilities are relatively close.")
        decision = "VERIFY_WITH_CUSTOMER"
        risk_level = "MEDIUM"

    # Generic ambiguity only when confidence is below 80%
    ambiguity_terms = [
        "issue", "problem", "something wrong", "not working",
        "what happened", "why", "help me", "help with"
    ]
    if top1 < 0.80 and any(term in q for term in ambiguity_terms):
        flags.append("AMBIGUOUS_LANGUAGE")
        reasons.append("The query uses broad or ambiguous wording.")
        if decision != "HUMAN_ESCALATION":
            decision = "VERIFY_WITH_CUSTOMER"
            risk_level = "MEDIUM"

    # Refined multiple-issue detection
    multi_markers = [
        "also", "as well as", "another issue", "another problem",
        "plus", ";"
    ]
    banking_signals = [
        "transfer", "payment", "card", "cash", "withdraw",
        "refund", "top up", "deposit", "account", "fee",
        "exchange", "verification"
    ]
    signal_count = sum(1 for term in banking_signals if term in q)

    if any(marker in q for marker in multi_markers) and signal_count >= 2:
        flags.append("POSSIBLE_MULTIPLE_ISSUES")
        reasons.append("The query may contain more than one banking issue.")
        decision = "VERIFY_WITH_CUSTOMER"
        risk_level = "MEDIUM"

    # Short-query protection
    if len(q.split()) <= 3 and top1 < 0.80:
        flags.append("SHORT_QUERY")
        reasons.append("The query is very short and may lack context.")
        if decision != "HUMAN_ESCALATION":
            decision = "VERIFY_WITH_CUSTOMER"
            risk_level = "MEDIUM"

    # Refined semantic safety guard
    direct_debit_explicit = (
        "direct debit" in q
        and (
            "don't recognize" in q
            or "do not recognize" in q
            or "not recognize" in q
            or "unrecognized" in q
            or "unauthorized" in q
            or "unknown" in q
        )
    )

    if (
        "debit card" in q
        and predicted_category == "direct_debit_payment_not_recognised"
        and not direct_debit_explicit
    ):
        flags.append("SEMANTIC_SAFETY_GUARD")
        reasons.append(
            "The wording refers to a debit card but does not explicitly "
            "indicate an unrecognized direct debit."
        )
        decision = "VERIFY_WITH_CUSTOMER"
        risk_level = "MEDIUM"

    escalate = decision == "HUMAN_ESCALATION"
    verification = decision == "VERIFY_WITH_CUSTOMER"

    return {
        "risk_level": risk_level,
        "decision": decision,
        "escalate": escalate,
        "verification_required": verification,
        "risk_flags": flags,
        "risk_reasons": reasons,
        "top2_margin": margin,
    }


def assess_risk(query, result):
    """
    Use the project's 08b risk module when it exposes a public
    assessment function. A signature adapter keeps this app robust
    across the exact saved Step 8B implementation.
    """
    top1 = confidence_from_result(result)
    top2 = top2_confidence_from_result(result)
    category = result.get("predicted_category", "")
```

```
    if risk is not None:
        candidate_names = [
            "assess_risk",
            "analyze_risk",
            "evaluate_risk",
            "assess_query",
            "analyze_query",
            "process_query",
        ]

        for name in candidate_names:
            fn = getattr(risk, name, None)
            if not callable(fn):
                continue

            try:
                sig = inspect.signature(fn)
                params = sig.parameters

                kwargs = {}

                mapping = {
                    "query": query,
                    "text": query,
                    "customer_query": query,
                    "predicted_category": category,
                    "category": category,
                    "confidence": top1,
                    "top1_confidence": top1,
                    "top2_confidence": top2,
                    "top1": top1,
                    "top2": top2,
                    "top_predictions": result.get("top_predictions", []),
                }

                required_missing = False
                positional = []

                for p in params.values():
                    if p.kind in (
                        inspect.Parameter.VAR_POSITIONAL,
                        inspect.Parameter.VAR_KEYWORD,
                    ):
                        continue

                    if p.name in mapping:
                        kwargs[p.name] = mapping[p.name]
                    elif p.default is inspect.Parameter.empty:
                        required_missing = True
                        break

                if not required_missing:
                    output = fn(**kwargs)
                    if isinstance(output, dict):
                        return output

                # Common positional fallbacks
                if len(params) == 4:
                    output = fn(query, category, top1, top2)
                    if isinstance(output, dict):
                        return output

                if len(params) == 3:
                    output = fn(query, category, top1)
                    if isinstance(output, dict):
                        return output

            except Exception:
                continue

    return fallback_refined_risk(query, category, top1, top2)


def normalize_risk_result(risk_result, result):
    """Ensure UI always receives the fields used below."""
    output = dict(risk_result or {})

    decision = output.get("decision", "")
    if decision not in {
        "AUTOMATED_RESPONSE",
        "VERIFY_WITH_CUSTOMER",
        "HUMAN_ESCALATION",
    }:
        decision = "HUMAN_ESCALATION" if confidence_from_result(result) < 0.60 else (
            "VERIFY_WITH_CUSTOMER"
            if confidence_from_result(result) < 0.80
            else "AUTOMATED_RESPONSE"
        )

    output["decision"] = decision
    output["escalate"] = bool(
        output.get("escalate", decision == "HUMAN_ESCALATION")
    )
```

```
    output["verification_required"] = bool(
        output.get(
            "verification_required",
            decision == "VERIFY_WITH_CUSTOMER",
        )
    )
    output["risk_level"] = output.get(
        "risk_level",
        "HIGH"
        if decision == "HUMAN_ESCALATION"
        else "MEDIUM"
        if decision == "VERIFY_WITH_CUSTOMER"
        else "LOW",
    )
    output["risk_flags"] = list(output.get("risk_flags", []))
    output["risk_reasons"] = list(output.get("risk_reasons", []))
    output["top2_margin"] = float(output.get("top2_margin", 0.0))
    return output


def build_safe_response(result, risk_result):
    """
    The risk layer is authoritative for whether the generated
    response may be treated as automated.
    """
    decision = risk_result["decision"]

    if decision == "HUMAN_ESCALATION":
        return (
            "I'm sorry you're experiencing this. I'm not confident enough "
            "to safely determine the exact banking issue from the information "
            "provided. I recommend routing this query to a human support "
            "specialist for review."
        )

    if decision == "VERIFY_WITH_CUSTOMER":
        return (
            "I want to make sure I understand your issue correctly before "
            "giving you specific next steps. Please confirm the exact "
            "transaction, payment, card issue, or banking service you are "
            "referring to."
        )

    return result.get(
        "generated_response",
        "I can help with your banking support query.",
    )


# ============================================================
# 6. HERO HEADER
# ============================================================

st.markdown(
    """
    <div class="hero">
        <div class="hero-title">
            ■ AI-Powered Banking Customer Support
        </div>
        <div class="hero-subtitle">
            Intelligent 77-class banking intent classification using
            fine-tuned DistilBERT, confidence-aware risk detection,
            human escalation, and safe AI-assisted customer support.
        </div>
    </div>
    """,
    unsafe_allow_html=True,
)


# ============================================================
# 7. SIDEBAR
# ============================================================

with st.sidebar:
    st.markdown(
        """
        <div style="text-align:center;padding:0.4rem 0 1rem 0;">
            <div style="font-size:3.2rem;">■</div>
            <div style="font-size:1.3rem;font-weight:850;">
                Banking AI
            </div>
            <div style="font-size:0.82rem;opacity:0.82;margin-top:0.3rem;">
                Customer Support Intelligence
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )

    st.divider()

    st.markdown("### ■ AI Model")
```

```
    st.write("**Model:** DistilBERT")
    st.write("**Classes:** 77")
    st.write("**Task:** Intent Classification")
    st.write("**Runtime:** CPU")

    st.divider()

    st.markdown("### ■ Confidence Policy")
    st.markdown("■ **HIGH** — ≥ 80%")
    st.markdown("■ **MEDIUM** — 60% to < 80%")
    st.markdown("■ **LOW** — < 60%")

    st.divider()

    st.markdown("### ■■ Risk Layer")
    st.markdown("■ **LOW RISK** — automated handling")
    st.markdown("■ **MEDIUM RISK** — verify customer")
    st.markdown("■ **HIGH RISK** — human escalation")

    st.divider()

    st.markdown("### ■ AI Workflow")
    st.markdown(
        """
        Customer Query
        ↓
        DistilBERT
        ↓
        Intent + Confidence
        ↓
        Risk & Ambiguity Detection
        ↓
        Escalation Decision
        ↓
        Safe AI Response

        """
    )


# ============================================================
# 8. MAIN TABS
# ============================================================

assistant_tab, analytics_tab = st.tabs(
    ["■ Support Assistant", "■ Model Analytics"]
)


# ============================================================
# 9. SUPPORT ASSISTANT
# ============================================================

with assistant_tab:
    st.markdown(
        '<div class="section-title">■ Customer Query</div>',
        unsafe_allow_html=True,
    )

    query = st.text_area(
        "Describe the customer's banking issue",
        height=150,
        placeholder=(
            "Example: I don't recognize a direct debit payment "
            "from my account."
        ),
        label_visibility="collapsed",
    )

    analyze = st.button(
        "■  Analyze Customer Query",
        use_container_width=True,
    )

    if analyze:
        if not query.strip():
            st.warning("Please enter a banking customer query.")
            st.stop()

        with st.spinner("■ Analyzing customer query..."):
            try:
                result = ai.process_customer_query(query.strip())
                risk_result = normalize_risk_result(
                    assess_risk(query.strip(), result),
                    result,
                )
            except Exception as e:
                st.error("An error occurred while processing the query.")
                st.exception(e)
                st.stop()

        # ----------------------------------------------------
        # QUERY
```

```
# ------------------------------------------------------

st.markdown(
    '<div class="section-title">■ Query Submitted</div>',
    unsafe_allow_html=True,
)

safe_query = html.escape(str(result.get("query", query.strip())))

st.markdown(
    f"""
    <div class="query-card">
        <strong>Customer:</strong><br>
        {safe_query}
    </div>
    """,
    unsafe_allow_html=True,
)

# ------------------------------------------------------
# FINAL OPERATIONAL DECISION
# ------------------------------------------------------

confidence_level = result.get("confidence_level", "LOW")
priority = get_priority(confidence_level)

decision = risk_result["decision"]

if decision == "AUTOMATED_RESPONSE":
    final_action = "AUTOMATED_RESPONSE"
    final_escalate = False
    final_verify = False
elif decision == "VERIFY_WITH_CUSTOMER":
    final_action = "VERIFY_WITH_CUSTOMER"
    final_escalate = False
    final_verify = True
else:
    final_action = "HUMAN_ESCALATION"
    final_escalate = True
    final_verify = False

# ------------------------------------------------------
# CLASSIFICATION
# ------------------------------------------------------

st.markdown(
    '<div class="section-title">■ AI Classification</div>',
    unsafe_allow_html=True,
)

c1, c2, c3, c4 = st.columns(4)

with c1:
    st.markdown(
        f"""
        <div class="info-card">
            <div class="card-icon">■</div>
            <div class="card-label">Predicted Intent</div>
            <div class="card-value">
                {html.escape(str(result["predicted_category"]))}
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )

with c2:
    st.markdown(
        f"""
        <div class="info-card">
            <div class="card-icon">■</div>
            <div class="card-label">Confidence</div>
            <div class="card-value">
                {float(result["confidence_percent"]):.2f}%
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )

with c3:
    st.markdown(
        f"""
        <div class="info-card">
            <div class="card-icon">■</div>
            <div class="card-label">Confidence Level</div>
            <div class="card-value">
                {get_confidence_badge(confidence_level)}
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )
```

```
with c4:
    st.markdown(
        f"""
        <div class="info-card">
            <div class="card-icon">■</div>
            <div class="card-label">Priority</div>
            <div class="card-value">
                {get_priority_badge(priority)}
            </div>
        </div>
        """,
        unsafe_allow_html=True,
    )

# ------------------------------------------------------
# RISK & ESCALATION
# ------------------------------------------------------

st.markdown(
    '<div class="section-title">■■ Risk & Escalation Decision</div>',
    unsafe_allow_html=True,
)

if decision == "AUTOMATED_RESPONSE":
    st.success(
        "■ LOW RISK — controlled automated handling is allowed."
    )
elif decision == "VERIFY_WITH_CUSTOMER":
    st.warning(
        "■ MEDIUM RISK — customer intent should be verified "
        "before automated handling."
    )
else:
    st.error(
        "■ HIGH RISK — human support escalation is recommended."
    )

d1, d2, d3 = st.columns(3)

with d1:
    st.write(f"**Final Action:** {final_action}")

with d2:
    st.write(
        f"**Escalation:** "
        f"{'Required' if final_escalate else 'Not Required'}"
    )

with d3:
    st.write(
        f"**Verification:** "
        f"{'Required' if final_verify else 'Not Required'}"
    )

flags = risk_result.get("risk_flags", [])
reasons = risk_result.get("risk_reasons", [])

if flags:
    flag_text = ", ".join(flags)
    reason_text = (
        "<br>".join(
            f"• {html.escape(str(x))}" for x in reasons
        )
        if reasons
        else "No additional explanation was returned."
    )

    st.markdown(
        f"""
        <div class="risk-card">
            <strong>Risk flags:</strong> {html.escape(flag_text)}
            <br><br>
            <strong>Why this was flagged:</strong><br>
            {reason_text}
        </div>
        """,
        unsafe_allow_html=True,
    )
else:
    st.markdown(
        """
        <div class="safe-card">
            <strong>Risk layer:</strong>
            No additional ambiguity or semantic-safety flags
            were detected for this query.
        </div>
        """,
        unsafe_allow_html=True,
    )

# ------------------------------------------------------
# TOP 3
```

```
# ----------------------------------------------------

st.markdown(
    '<div class="section-title">■ Top-3 Intent Predictions</div>',
    unsafe_allow_html=True,
)

for item in result.get("top_predictions", [])[:3]:
    left, right = st.columns([5, 1])

    with left:
        st.write(
            f"**{item['rank']}. {item['category']}**"
        )
        st.progress(
            min(float(item["confidence"]), 1.0)
        )

    with right:
        st.markdown(
            f"**{float(item['confidence_percent']):.2f}%**"
        )

# ----------------------------------------------------
# AI RESPONSE
# ----------------------------------------------------

st.markdown(
    '<div class="section-title">■ AI Customer Support Response</div>',
    unsafe_allow_html=True,
)

if final_action == "AUTOMATED_RESPONSE":
    st.success("Automated response")
elif final_action == "VERIFY_WITH_CUSTOMER":
    st.warning("Verification response")
else:
    st.error("Escalation response")

final_response = build_safe_response(result, risk_result)
safe_response = html.escape(str(final_response))

st.markdown(
    f"""
    <div class="response-card">
        {safe_response}
    </div>
    """,
    unsafe_allow_html=True,
)

# ----------------------------------------------------
# SAFETY
# ----------------------------------------------------

st.markdown(
    '<div class="section-title">■■ AI Safety Check</div>',
    unsafe_allow_html=True,
)

if final_action == "AUTOMATED_RESPONSE" and result.get(
    "safety_check", False
):
    st.success(
        "■ Safety check passed — no customer-specific account "
        "facts were invented."
    )
elif final_action != "AUTOMATED_RESPONSE":
    st.info(
        "■■ The generated response has been replaced by a "
        "conservative verification/escalation response because "
        "the risk layer did not approve direct automation."
    )
else:
    st.error("■■ Safety check failed.")

# ----------------------------------------------------
# SUPPORT SUMMARY
# ----------------------------------------------------

st.markdown(
    '<div class="section-title">■ Support Summary</div>',
    unsafe_allow_html=True,
)

summary_col1, summary_col2 = st.columns(2)

with summary_col1:
    st.write(f"**Customer Issue:** {result['query']}")
    st.write(
        f"**Detected Intent:** {result['predicted_category']}"
    )
```

```
            st.write(f"**Priority:** {priority}")

        with summary_col2:
            st.write(
                f"**Confidence:** {float(result['confidence_percent']):.2f}%"

            )
            st.write(f"**Final Action:** {final_action}")
            st.write(
                f"**Risk Level:** {risk_result['risk_level']}"
            )

        # ----------------------------------------------------
        # TECHNICAL DETAILS
        # ----------------------------------------------------

        with st.expander("■ View Technical Prediction Details"):
            st.json(
                {
                    "model_result": result,
                    "risk_result": risk_result,
                    "final_action": final_action,
                    "final_escalate": final_escalate,
                    "final_verification": final_verify,
                }
            )


# ============================================================
# 10. MODEL ANALYTICS
# ============================================================

with analytics_tab:
    st.markdown(
        '<div class="section-title">■ Model Performance Analytics</div>',
        unsafe_allow_html=True,
    )

    st.info(
        "These metrics come from the completed official final test "
        "evaluation. The official test set was not used for model training."
    )

    metrics = load_final_metrics()

    # Completed official final evaluation values
    transformer_accuracy = 88.67
    transformer_f1 = 88.16
    ml_accuracy = 87.79
    ml_f1 = 87.87
    accuracy_improvement = 0.88
    f1_improvement = 0.28

    a1, a2, a3, a4 = st.columns(4)

    with a1:
        st.metric("DistilBERT Accuracy", "88.67%")

    with a2:
        st.metric("DistilBERT Weighted F1", "88.16%")

    with a3:
        st.metric("ML Baseline Accuracy", "87.79%")

    with a4:
        st.metric("Accuracy Improvement", "+0.88 pp")

    st.markdown(
        '<div class="section-title">■■ ML Baseline vs Transformer</div>',
        unsafe_allow_html=True,
    )

    comparison_data = {
        "Metric": ["Accuracy", "Weighted F1"],
        "TF-IDF + Logistic Regression": ["87.79%", "87.87%"],
        "DistilBERT": ["88.67%", "88.16%"],
        "Improvement": ["+0.88 pp", "+0.28 pp"],
    }

    st.table(comparison_data)

    st.markdown(
        '<div class="section-title">■ Confidence Distribution</div>',
        unsafe_allow_html=True,
    )

    confidence_col1, confidence_col2, confidence_col3 = st.columns(3)

    with confidence_col1:
        st.metric("■ High Confidence", "57.03%")
        st.progress(0.5703)

    with confidence_col2:
```

```
    st.metric("■ Medium Confidence", "22.28%")
    st.progress(0.2228)

with confidence_col3:
    st.metric("■ Low Confidence", "20.69%")
    st.progress(0.2069)

st.markdown(
    '<div class="section-title">■ Confidence Statistics</div>',
    unsafe_allow_html=True,
)

s1, s2, s3, s4 = st.columns(4)

with s1:
    st.metric("Mean Confidence", "75.32%")

with s2:
    st.metric("Median Confidence", "83.27%")

with s3:
    st.metric("Minimum", "10.86%")

with s4:
    st.metric("Maximum", "95.86%")

st.markdown(
    '<div class="section-title">■■ Refined Risk Policy</div>',
    unsafe_allow_html=True,
)

policy_col1, policy_col2, policy_col3 = st.columns(3)

with policy_col1:
    st.success(
        """
        **LOW RISK**

        High-confidence predictions can proceed with controlled
        automated handling when no semantic or ambiguity flags are
        triggered.
        """
    )

with policy_col2:
    st.warning(
        """
        **MEDIUM RISK**

        Verify the customer's intent when confidence is moderate,
        competing intents are close, or a semantic safety rule fires.
        """
    )

with policy_col3:
    st.error(
        """
        **HIGH RISK**

        Low-confidence queries should be routed to human support.
        """
    )

st.markdown(
    '<div class="section-title">■ Error-Analysis Findings</div>',
    unsafe_allow_html=True,
)

st.write(
    """
    The completed official-test error analysis found that the model
    achieved 88.67% accuracy, with 2,730 correct and 349 incorrect
    predictions across 3,079 official test rows.

    The weakest observed class was **virtual_card_not_working**,
    where the model confused examples with related virtual-card
    intents. Other notable confusion areas included
    **why_verify_identity vs verify_my_identity** and
    **get_disposable_virtual_card vs related disposable-card intents**.

    The confidence analysis also showed that high-confidence
    predictions are generally much more reliable than low-confidence
    predictions. However, the application therefore uses the refined

    risk layer as an additional safety check rather than treating
    confidence alone as proof of correctness.
    """
)

st.markdown(
    '<div class="section-title">■ Semantic Safety Example</div>',
    unsafe_allow_html=True,
)
```

```
    st.write(
        """
        The refined risk layer specifically protects against a known
        semantic mismatch such as **"issue with debit card"** being
        classified with high confidence as
        **direct_debit_payment_not_recognised**.

        In that situation, the application changes the operational
        decision to **VERIFY_WITH_CUSTOMER** instead of blindly sending
        an automated intent-specific response.
        """
    )

    st.markdown(
        '<div class="section-title">■■ Evaluation Note</div>',
        unsafe_allow_html=True,
    )

    st.write(
        """
        The final DistilBERT model achieved 88.67% accuracy and 88.16%
        weighted F1 on the official test set. The TF-IDF + Logistic
        Regression baseline achieved 87.79% accuracy and 87.87% weighted
        F1.

        The 80% / 60% confidence bands are the operational thresholds
        used in this application. The refined Step 8B risk layer adds
        ambiguity, close-prediction, multiple-issue, short-query and
        semantic-safety checks on top of the confidence bands.
        """
    )


# ============================================================
# 11. FOOTER
# ============================================================

st.markdown(
    """
    <div class="footer">
        ■ AI-Powered Banking Customer Support
          •  
        DistilBERT 77-Class Intent Classification
          •  
        Confidence & Risk-Aware AI
          •  
        Safe Customer Assistance
    </div>
    """,
    unsafe_allow_html=True,
)
```

20B. 09_project_requirement_audit_corrected.py — Complete Available Source

```
# ============================================================
# AI-POWERED BANKING CUSTOMER SUPPORT
# FINAL PROJECT REQUIREMENT AUDIT
# ============================================================
#
# Purpose:
#   Audit the CLEAN FINAL_PROJECT against the project
#   requirements and available evidence.
#
# Status values:
#   PASS     = sufficient evidence found
#   PARTIAL  = some evidence found, but manual verification needed
#   MISSING  = required evidence/file/result not found
#   N/A      = requirement is optional / not mandatory
#
# IMPORTANT:
#   This script DOES NOT:
#       - train a model
#       - modify the model
#       - modify datasets
#       - modify app.py
#       - evaluate the official test set again
#
#   It only reads project files and creates an audit report.
# ============================================================

import os
import json
import re
from datetime import datetime


# ============================================================
# 1. PROJECT PATHS
# ============================================================

BASE_DIR = os.path.dirname(
    os.path.dirname(os.path.abspath(__file__))
)

SRC_DIR = os.path.join(BASE_DIR, "src")
DATASET_DIR = os.path.join(BASE_DIR, "dataset")
RAW_DIR = os.path.join(DATASET_DIR, "raw")
PROCESSED_DIR = os.path.join(DATASET_DIR, "processed")

RESULTS_DIR = os.path.join(BASE_DIR, "results")
EVALUATION_DIR = os.path.join(RESULTS_DIR, "evaluation")
REPORTS_DIR = os.path.join(RESULTS_DIR, "reports")

MODEL_DIR = os.path.join(
    DATASET_DIR,
    "checkpoints_step3_exp2",
    "best_model"
)

APP_FILE = os.path.join(BASE_DIR, "app.py")

README_FILE = os.path.join(BASE_DIR, "README.md")
REQUIREMENTS_FILE = os.path.join(BASE_DIR, "requirements.txt")

AUDIT_OUTPUT = os.path.join(
    REPORTS_DIR,
    "project_requirement_audit.json"
)

AUDIT_TEXT_OUTPUT = os.path.join(
    REPORTS_DIR,
    "project_requirement_audit.txt"
)


# ============================================================
# 2. HELPERS
# ============================================================

results = []


def exists(path):
    return os.path.exists(path)


def read_text(path):
    if not exists(path):
        return ""

    try:
        with open(
            path,
            "r",
```

```
            encoding="utf-8",
            errors="ignore"
        ) as f:
            return f.read()

    except Exception:
        return ""


def add_result(
    requirement,
    status,
    evidence,
    details="",
    action=""
):
    results.append(
        {
            "requirement": requirement,
            "status": status,
            "evidence": evidence,
            "details": details,
            "recommended_action": action,
        }
    )


def count_files(folder):
    if not exists(folder):
        return 0

    total = 0

    for root, dirs, files in os.walk(folder):
        total += len(files)

    return total


def find_files(folder, extensions=None):
    found = []

    if not exists(folder):
        return found

    for root, dirs, files in os.walk(folder):

        for filename in files:

            if extensions:

                if not any(
                    filename.lower().endswith(ext.lower())
                    for ext in extensions
                ):
                    continue

            found.append(
                os.path.join(root, filename)
            )

    return found


def contains_any(text, terms):
    text_lower = text.lower()

    return any(
        term.lower() in text_lower
        for term in terms
    )


def contains_all(text, terms):
    text_lower = text.lower()

    return all(
        term.lower() in text_lower
        for term in terms
    )

def file_has_any(path, terms):
    text = read_text(path)

    if not text:
        return False

    return contains_any(text, terms)


def format_status(status):
    if status == "PASS":
        return "■ PASS"
```

```
    if status == "PARTIAL":
        return "■ PARTIAL"

    if status == "MISSING":
        return "■ MISSING"

    return "■ N/A"


# ============================================================
# 3. REQUIRED FINAL SOURCE FILES
# ============================================================

required_source_files = [
    "01_data_preprocessing.py",
    "02_machine_learning_baseline.py",
    "03_deep_learning_bert.py",
    "03b_deep_learning_exp2.py",
    "04_evaluate_model.py",
    "05_inference.py",
    "06_confidence_escalation.py",
    "07_generative_ai.py",
    "08_error_analysis.py",
    "08b_risk_detection.py",
]


# ============================================================
# 4. ROOT FILE CHECK
# ============================================================

root_files_ok = all(
    exists(os.path.join(BASE_DIR, filename))
    for filename in [
        "app.py",
        "README.md",
        "requirements.txt",
    ]
)

if root_files_ok:

    add_result(
        "Project structure / core files",
        "PASS",
        "app.py, README.md and requirements.txt",
        "Core final-project files are present."
    )

else:

    missing = [
        filename
        for filename in [
            "app.py",
            "README.md",
            "requirements.txt",
        ]
        if not exists(os.path.join(BASE_DIR, filename))
    ]

    add_result(
        "Project structure / core files",
        "MISSING",
        "Required root files",
        "Missing: " + ", ".join(missing),
        "Restore the missing required file(s)."
    )
# ============================================================
# 5. DATASET REQUIREMENT
# ============================================================

train_raw = os.path.join(
    RAW_DIR,
    "train.csv"
)

test_raw = os.path.join(
    RAW_DIR,
    "test.csv"
)

train_clean = os.path.join(
    PROCESSED_DIR,
    "train_clean.csv"
)

test_clean = os.path.join(
    PROCESSED_DIR,
    "test_clean.csv"
)
```

```
label_mapping = os.path.join(
    PROCESSED_DIR,
    "label_mapping.json"
)

dataset_files_ok = all(
    exists(path)
    for path in [
        train_raw,
        test_raw,
        train_clean,
        test_clean,
        label_mapping,
    ]
)

if dataset_files_ok:

    add_result(
        "Dataset and processed data",
        "PASS",
        (
            "dataset/raw/train.csv; "
            "dataset/raw/test.csv; "
            "dataset/processed/train_clean.csv; "
            "dataset/processed/test_clean.csv; "
            "label_mapping.json"
        ),
        "Raw and processed dataset artifacts are present."
    )

else:

    missing = []

    for path in [
        train_raw,
        test_raw,
        train_clean,
        test_clean,
        label_mapping,
    ]:

        if not exists(path):
            missing.append(
                os.path.relpath(path, BASE_DIR)
            )

    add_result(
        "Dataset and processed data",
        "MISSING",
        "Expected dataset artifacts",
        "Missing: " + ", ".join(missing),
        "Restore the missing dataset artifact(s)."
    )


# ============================================================
# 6. LABEL MAPPING / 77 CLASSES
# ============================================================

if exists(label_mapping):

    try:

        with open(
            label_mapping,
            "r",
            encoding="utf-8"
        ) as f:

            mapping = json.load(f)

        if isinstance(mapping, dict):

            # label_mapping.json has three top-level keys. The actual
            # intent count is stored in num_classes or in label2id/id2label.
            if isinstance(mapping.get("num_classes"), int):
                number_of_labels = mapping["num_classes"]
            elif isinstance(mapping.get("label2id"), dict):
                number_of_labels = len(mapping["label2id"])
            elif isinstance(mapping.get("id2label"), dict):
                number_of_labels = len(mapping["id2label"])
            else:
                number_of_labels = 0

        else:

            number_of_labels = 0

        if number_of_labels == 77:

            add_result(
```

```
                "77-category intent classification",
                "PASS",
                "dataset/processed/label_mapping.json",
                "Label mapping contains exactly 77 classes."
            )

        else:

            add_result(
                "77-category intent classification",
                "PARTIAL",
                "label_mapping.json",
                f"Detected {number_of_labels} labels instead of 77.",
                "Verify the official 77-class mapping."
            )

    except Exception as e:

        add_result(
            "77-category intent classification",
            "PARTIAL",
            "label_mapping.json",
            f"Could not parse mapping: {e}",
            "Verify label_mapping.json."
        )

else:

    add_result(
        "77-category intent classification",
        "MISSING",
        "label_mapping.json",
        "Label mapping was not found.",
        "Restore the official label mapping."
    )


# ============================================================
# 7. DATA PREPROCESSING
# ============================================================

preprocess_file = os.path.join(
    SRC_DIR,
    "01_data_preprocessing.py"
)

preprocess_text = read_text(preprocess_file)

preprocessing_terms = [
    "dropna",
    "duplicate",
    "text",
    "category",
    "label",
]

if exists(preprocess_file) and contains_all(
    preprocess_text,
    preprocessing_terms
):

    add_result(
        "NLP / data preprocessing",
        "PASS",
        "src/01_data_preprocessing.py",
        "Preprocessing script contains text/category handling, duplicate and missing-value handling."
    )

elif exists(preprocess_file):

    add_result(
        "NLP / data preprocessing",
        "PARTIAL",
        "src/01_data_preprocessing.py",
        "Preprocessing script exists but not all expected preprocessing evidence was detected.",
        "Review preprocessing implementation manually."
    )

else:

    add_result(
        "NLP / data preprocessing",
        "MISSING",
        "src/01_data_preprocessing.py",
        "Preprocessing script not found."
    )


# ============================================================
# 8. EDA
# ============================================================
```

```
eda_evidence = []

report_files = find_files(
    REPORTS_DIR
)

for path in report_files:

    text = read_text(path)

    if contains_any(
        text,
        [
            "category",
            "text_length",
            "duplicate",
            "missing",
            "distribution",
        ]
    ):

        eda_evidence.append(
            os.path.relpath(path, BASE_DIR)
        )


if eda_evidence:

    add_result(
        "Exploratory Data Analysis",
        "PASS",
        "; ".join(eda_evidence[:5]),
        "Dataset statistics / EDA evidence was found."
    )

else:

    notebooks = find_files(
        os.path.join(BASE_DIR, "notebooks"),
        [".ipynb"]
    )

    if notebooks:

        add_result(
            "Exploratory Data Analysis",

            "PARTIAL",
            "; ".join(
                os.path.relpath(
                    x,
                    BASE_DIR
                )
                for x in notebooks
            ),
            "Notebook evidence exists, but EDA result artifacts were not automatically confirmed.",
            "Open the EDA notebook and verify required analyses."
        )

    else:

        add_result(
            "Exploratory Data Analysis",
            "MISSING",
            "results/reports or notebooks",
            "No strong EDA evidence was detected.",
            "Include EDA evidence in the final project."
        )


# ============================================================
# 9. ML BASELINE
# ============================================================

ml_file = os.path.join(
    SRC_DIR,
    "02_machine_learning_baseline.py"
)

ml_text = read_text(ml_file)

ml_terms = [
    "TfidfVectorizer",
    "LogisticRegression",
]

ml_results_files = find_files(
    os.path.join(
        RESULTS_DIR,
        "ml"
    )
)
```

```
if exists(ml_file) and contains_all(
    ml_text,
    ml_terms
):

    if ml_results_files:

        add_result(
            "ML baseline",
            "PASS",
            (
                "src/02_machine_learning_baseline.py; "
                "results/ml"
            ),
            "TF-IDF + Logistic Regression baseline and result artifacts found."
        )

    else:

        add_result(
            "ML baseline",
            "PARTIAL",
            "src/02_machine_learning_baseline.py",
            "Baseline implementation found but result artifacts were not detected.",
            "Verify and preserve ML baseline metrics."
        )

else:

    add_result(
        "ML baseline",
        "MISSING",
        "src/02_machine_learning_baseline.py",
        "TF-IDF + Logistic Regression evidence not detected.",
        "Restore or verify the ML baseline."
    )


# ============================================================

# 10. DISTILBERT / DEEP LEARNING
# ============================================================

bert_file = os.path.join(
    SRC_DIR,
    "03b_deep_learning_exp2.py"
)

bert_text = read_text(bert_file)

bert_terms = [
    "DistilBertTokenizerFast",
    "DistilBertForSequenceClassification",
    "distilbert-base-uncased",
    "num_labels",
]

bert_exists = exists(bert_file)

model_exists = exists(MODEL_DIR)

if bert_exists and contains_all(
    bert_text,
    bert_terms
) and model_exists:

    add_result(
        "Deep learning / DistilBERT",
        "PASS",
        (
            "src/03b_deep_learning_exp2.py; "
            "dataset/checkpoints_step3_exp2/best_model"
        ),
        "DistilBERT training implementation and final best model are present."
    )

elif bert_exists and model_exists:

    add_result(
        "Deep learning / DistilBERT",
        "PARTIAL",
        (
            "03b_deep_learning_exp2.py; "
            "best_model"
        ),
        "Training script and model exist, but expected transformer implementation terms were not all detected.",
        "Review the final training script."
    )

else:

    add_result(
        "Deep learning / DistilBERT",
```

```
        "MISSING",
        "DistilBERT training script / best_model",
        "Final transformer implementation or model is missing.",
        "Restore the final DistilBERT model and script."
    )


# ============================================================
# 11. OFFICIAL EVALUATION
# ============================================================

evaluation_file = os.path.join(
    SRC_DIR,
    "04_evaluate_model.py"
)

evaluation_text = read_text(evaluation_file)

evaluation_terms = [
    "accuracy",
    "precision",
    "recall",
    "f1",
    "confusion",
    "confidence",
]

evaluation_results = find_files(
    EVALUATION_DIR
)

if exists(evaluation_file) and contains_all(
    evaluation_text,
    evaluation_terms
) and evaluation_results:

    add_result(
        "Official model evaluation",
        "PASS",
        (
            "src/04_evaluate_model.py; "
            "results/evaluation"
        ),
        "Evaluation code and final evaluation artifacts are present."
    )

elif exists(evaluation_file):

    add_result(
        "Official model evaluation",
        "PARTIAL",
        "src/04_evaluate_model.py",
        "Evaluation script exists but complete evidence was not automatically confirmed.",
        "Verify accuracy, precision, recall, F1, confusion matrix and confidence outputs."
    )

else:

    add_result(
        "Official model evaluation",
        "MISSING",
        "src/04_evaluate_model.py",
        "Official evaluation script was not found."
    )


# ============================================================
# 12. CONFIDENCE ANALYSIS
# ============================================================

confidence_files = find_files(
    RESULTS_DIR
)

confidence_evidence = []

for path in confidence_files:

    filename = os.path.basename(path).lower()

    text = read_text(path).lower()

    if (
        "confidence" in filename
        or "confidence" in text
    ):

        confidence_evidence.append(
            os.path.relpath(path, BASE_DIR)
        )

if confidence_evidence:
```

```
    add_result(
        "Prediction confidence",
        "PASS",
        "; ".join(confidence_evidence[:8]),
        "Confidence-related evidence was found in the final results."
    )

else:

    add_result(
        "Prediction confidence",
        "MISSING",
        "results/",
        "No confidence-analysis evidence detected.",
        "Preserve final confidence metrics and analysis."
    )


# ============================================================
# 13. CONFIDENCE / ESCALATION POLICY
# ============================================================

confidence_script = os.path.join(
    SRC_DIR,
    "06_confidence_escalation.py"
)

confidence_script_text = read_text(
    confidence_script
)

escalation_terms = [
    "HUMAN_ESCALATION",
    "VERIFY_WITH_CUSTOMER",
    "AUTOMATED_RESPONSE",
]

if exists(confidence_script) and contains_all(
    confidence_script_text,
    escalation_terms
):

    add_result(
        "Confidence-based escalation",
        "PASS",
        "src/06_confidence_escalation.py",
        "Automated, verification and human-escalation decisions are represented."
    )

elif exists(confidence_script):

    add_result(
        "Confidence-based escalation",
        "PARTIAL",
        "src/06_confidence_escalation.py",
        "Escalation script exists but all operational decision states were not detected.",
        "Review the escalation policy."
    )

else:

    add_result(
        "Confidence-based escalation",
        "MISSING",
        "src/06_confidence_escalation.py",
        "Escalation script not found."
    )


# ============================================================
# 14. RISK / AMBIGUITY DETECTION
# ============================================================

risk_file = os.path.join(
    SRC_DIR,
    "08b_risk_detection.py"
)

risk_text = read_text(risk_file)

risk_terms = [
    "AMBIGUOUS_LANGUAGE",
    "CLOSE_TOP_PREDICTIONS",
    "LOW_CONFIDENCE",
    "SEMANTIC_SAFETY_GUARD",
]

if exists(risk_file) and contains_all(
    risk_text,
    risk_terms
):
```

```
    add_result(
        "Refined risk and ambiguity detection",
        "PASS",
        "src/08b_risk_detection.py",
        "Refined ambiguity, confidence and semantic-safety controls are present."
    )

elif exists(risk_file):

    add_result(
        "Refined risk and ambiguity detection",
        "PARTIAL",
        "src/08b_risk_detection.py",
        "Risk module exists but not all expected safeguards were detected.",
        "Review the final Step 8B implementation."
    )
else:

    add_result(
        "Refined risk and ambiguity detection",
        "MISSING",
        "src/08b_risk_detection.py",
        "Risk detection module not found."
    )


# ============================================================
# 15. GENERATIVE AI
# ============================================================

genai_file = os.path.join(
    SRC_DIR,
    "07_generative_ai.py"
)

genai_text = read_text(
    genai_file
)

genai_terms = [
    "process_customer_query",
    "generated_response",
    "confidence",
]

if exists(genai_file) and contains_all(
    genai_text,
    genai_terms
):

    add_result(
        "Generative AI response",
        "PASS",
        "src/07_generative_ai.py",
        "Generative/support-response pipeline is present."
    )

elif exists(genai_file):

    add_result(
        "Generative AI response",
        "PARTIAL",
        "src/07_generative_ai.py",
        "GenAI module exists but complete response evidence was not automatically confirmed.",
        "Manually verify response generation."
    )

else:

    add_result(
        "Generative AI response",
        "MISSING",
        "src/07_generative_ai.py",
        "Generative AI module not found."
    )


# ============================================================
# 16. SAFETY / HALLUCINATION CONTROL
# ============================================================

app_text = read_text(APP_FILE)

safety_terms = [
    "build_safe_response",
    "safety_check",
    "HUMAN_ESCALATION",
    "VERIFY_WITH_CUSTOMER",
]

if exists(APP_FILE) and contains_all(
    app_text,
```

```
    safety_terms
):

    add_result(
        "AI safety / controlled response",
        "PASS",
        "app.py",
        "Safe-response and risk-controlled handling are implemented in the application."
    )

elif exists(APP_FILE):

    add_result(
        "AI safety / controlled response",
        "PARTIAL",
        "app.py",
        "Application exists but complete safety evidence was not detected.",
        "Manually test low-confidence and semantically risky queries."
    )

else:

    add_result(
        "AI safety / controlled response",
        "MISSING",
        "app.py",
        "Final Streamlit application was not found."
    )


# ============================================================
# 17. CONVERSATION INTELLIGENCE / SUPPORT SUMMARY
# ============================================================

conversation_terms = [
    "Support Summary",
    "Customer Issue",
    "Detected Intent",
    "Priority",
    "Final Action",
]

if exists(APP_FILE) and contains_all(
    app_text,
    conversation_terms
):

    add_result(
        "Conversation intelligence / support summary",
        "PASS",
        "app.py",
        "Support summary includes issue, intent, priority and action."
    )

elif exists(APP_FILE):

    add_result(
        "Conversation intelligence / support summary",
        "PARTIAL",
        "app.py",
        "Application exists but complete support-summary evidence was not detected.",
        "Verify the support summary manually."
    )

else:

    add_result(
        "Conversation intelligence / support summary",
        "MISSING",
        "app.py",
        "Application missing."
    )


# ============================================================
# 18. STREAMLIT APPLICATION
# ============================================================

streamlit_terms = [
    "st.set_page_config",
    "st.text_area",
    "st.button",
    "st.tabs",
    "Model Performance Analytics",
]

if exists(APP_FILE) and contains_all(
    app_text,
    streamlit_terms
):

    add_result(
```

```
        "Streamlit application",
        "PASS",
        "app.py",
        "Streamlit interface, query input, interaction and analytics tabs detected."
    )

elif exists(APP_FILE):

    add_result(
        "Streamlit application",
        "PARTIAL",
        "app.py",
        "Streamlit application exists but complete UI evidence was not detected.",
        "Run and manually verify the application."
    )

else:

    add_result(
        "Streamlit application",
        "MISSING",
        "app.py",
        "Final Streamlit application missing."
    )


# ============================================================
# 19. STREAMLIT REQUIRED OUTPUTS
# ============================================================

ui_terms = [
    "Predicted Intent",
    "Confidence",
    "Priority",
    "Risk & Escalation Decision",
    "AI Customer Support Response",
    "Support Summary",
    "Model Performance Analytics",
]

if exists(APP_FILE) and contains_all(
    app_text,
    ui_terms
):

    add_result(
        "Required Streamlit outputs",
        "PASS",
        "app.py",
        "Required prediction, confidence, priority, response, escalation, summary and analytics sections detected."
    )

elif exists(APP_FILE):

    add_result(
        "Required Streamlit outputs",
        "PARTIAL",
        "app.py",
        "Some required UI sections were not automatically detected.",
        "Review the Streamlit interface manually."
    )

else:

    add_result(
        "Required Streamlit outputs",
        "MISSING",
        "app.py",
        "Application missing."
    )


# ============================================================
# 20. ML VS TRANSFORMER COMPARISON
# ============================================================

comparison_terms = [
    "TF-IDF + Logistic Regression",
    "DistilBERT",
    "87.79%",
    "88.67%",
]

if exists(APP_FILE) and contains_all(
    app_text,
    comparison_terms
):

    add_result(
        "ML vs Transformer comparison",
        "PASS",
```

```
        "app.py",
        "Final application contains the ML baseline vs DistilBERT comparison."
    )

else:

    comparison_files = find_files(
        RESULTS_DIR
    )

    found_comparison = False

    for path in comparison_files:

        text = read_text(path)

        if contains_all(
            text,
            [
                "87.79",
                "88.67",
            ]
        ):

            found_comparison = True
            break

    if found_comparison:

        add_result(
            "ML vs Transformer comparison",
            "PASS",
            "results/",
            "Comparison metrics were found in result artifacts."
        )

    else:

        add_result(
            "ML vs Transformer comparison",
            "MISSING",
            "app.py / results/",
            "Final ML vs Transformer comparison was not detected.",
            "Preserve the final comparison metrics."
        )


# ============================================================
# 21. ERROR ANALYSIS
# ============================================================

error_file = os.path.join(
    SRC_DIR,
    "08_error_analysis.py"
)

error_results = find_files(
    os.path.join(
        RESULTS_DIR,
        "error_analysis"
    )
)

if exists(error_file) and error_results:

    add_result(
        "Error analysis",
        "PASS",
        (
            "src/08_error_analysis.py; "
            "results/error_analysis"
        ),
        "Error-analysis script and output artifacts are present."
    )

elif exists(error_file):

    add_result(
        "Error analysis",
        "PARTIAL",

        "src/08_error_analysis.py",
        "Error-analysis code exists but result artifacts were not found.",
        "Preserve the final error-analysis outputs."
    )

else:

    add_result(
        "Error analysis",
        "MISSING",
        "src/08_error_analysis.py",
        "Error-analysis script missing."
```

```
    )


# ============================================================
# 22. DOCUMENTATION
# ============================================================

if exists(README_FILE):

    readme_text = read_text(
        README_FILE
    )

    documentation_terms = [
        "DistilBERT",
        "Streamlit",
        "dataset",
        "evaluation",
        "Project Workflow",
        "Machine Learning Baseline",
        "Deep Learning Model",
        "Prediction Confidence",
        "Escalation Strategy",
        "Generative AI",
        "AI Safety",
        "Project Structure",
        "Installation",
        "Running the Application",
        "Final Results",
        "Limitations",
    ]

    matched = sum(
        1
        for term in documentation_terms
        if term.lower() in readme_text.lower()
    )

    if matched >= 8:

        add_result(
            "Project documentation",
            "PASS",
            "README.md",
            "README contains substantial project documentation."
        )

    else:

        add_result(
            "Project documentation",
            "PARTIAL",
            "README.md",
            "README exists but appears incomplete.",
            "Add workflow, setup, model, evaluation and usage documentation."
        )

else:

    add_result(
        "Project documentation",
        "MISSING",
        "README.md",
        "README file not found.",
        "Create the final project README."
    )


# ============================================================
# 23. REQUIREMENTS FILE
# ============================================================

if exists(REQUIREMENTS_FILE):

    requirements_text = read_text(
        REQUIREMENTS_FILE
    )

    package_terms = [
        "streamlit",
        "torch",
        "transformers",
    ]

    matched = sum(
        1
        for term in package_terms
        if term.lower() in requirements_text.lower()
    )

    if matched >= 2:

        add_result(
```

```
            "Environment / requirements",
            "PASS",
            "requirements.txt",
            "Core runtime dependencies are represented."
        )

    else:

        add_result(
            "Environment / requirements",
            "PARTIAL",
            "requirements.txt",
            "requirements.txt exists but core dependencies were not all detected.",
            "Verify the final dependency list."
        )

else:

    add_result(
        "Environment / requirements",
        "MISSING",
        "requirements.txt",
        "requirements.txt missing.",
        "Create the final dependency file."
    )


# ============================================================
# 24. OFFICIAL TEST SET PROTECTION
# ============================================================

evaluation_code = read_text(
    evaluation_file
)

official_test_evidence = (
    "test_clean.csv" in evaluation_code
    or "test.csv" in evaluation_code
)

training_code = read_text(
    bert_file
)

training_uses_official_test = (
    "test_clean.csv" in training_code.lower()
    and (
        "train" in training_code.lower()
    )
)

if official_test_evidence and not training_uses_official_test:

    add_result(
        "Official test-set separation",
        "PASS",
        "03b_deep_learning_exp2.py and 04_evaluate_model.py",
        "Training/evaluation separation appears consistent with the final workflow."
    )

else:

    add_result(
        "Official test-set separation",
        "PARTIAL",

        "Training/evaluation scripts",
        "Automatic check could not conclusively verify test-set protection.",
        "Manually confirm official test.csv was reserved for final evaluation."
    )


# ============================================================
# 25. OPTIONAL DEPLOYMENT
# ============================================================

add_result(
    "Deployment",
    "N/A",
    "Project specification",
    "Deployment is optional according to the project specification."
)


# ============================================================
# 26. CALCULATE OVERALL STATUS
# ============================================================

status_counts = {
    "PASS": 0,
    "PARTIAL": 0,
    "MISSING": 0,
    "N/A": 0,
```

```
}

for item in results:

    status = item["status"]

    if status in status_counts:
        status_counts[status] += 1


mandatory_results = [
    item
    for item in results
    if item["status"] != "N/A"
]

mandatory_missing = sum(
    1
    for item in mandatory_results
    if item["status"] == "MISSING"
)

mandatory_partial = sum(
    1
    for item in mandatory_results
    if item["status"] == "PARTIAL"
)

mandatory_pass = sum(
    1
    for item in mandatory_results
    if item["status"] == "PASS"
)


if mandatory_missing > 0:

    overall_status = "NEEDS FIXES"

elif mandatory_partial > 0:

    overall_status = "NEEDS MANUAL VERIFICATION"

else:

    overall_status = "READY"


# ============================================================
# 27. CREATE OUTPUT DIRECTORY
# ============================================================

os.makedirs(
    REPORTS_DIR,
    exist_ok=True
)
# ============================================================
# 28. JSON REPORT
# ============================================================

audit_report = {
    "project": "AI-Powered Banking Customer Support",
    "audit_type": "Final Project Requirement Audit",
    "generated_at": datetime.now().isoformat(
        timespec="seconds"
    ),
    "project_path": BASE_DIR,
    "overall_status": overall_status,
    "summary": {
        "pass": status_counts["PASS"],
        "partial": status_counts["PARTIAL"],
        "missing": status_counts["MISSING"],
        "not_applicable": status_counts["N/A"],
    },
    "results": results,
}


with open(
    AUDIT_OUTPUT,
    "w",
    encoding="utf-8"
) as f:

    json.dump(
        audit_report,
        f,
        indent=4
    )


# ============================================================
```

```
# 29. TEXT REPORT
# ============================================================

with open(
    AUDIT_TEXT_OUTPUT,
    "w",
    encoding="utf-8"
) as f:

    f.write(
        "AI-POWERED BANKING CUSTOMER SUPPORT\n"
    )

    f.write(
        "FINAL PROJECT REQUIREMENT AUDIT\n"
    )

    f.write("=" * 70 + "\n\n")

    f.write(
        f"Overall Status: {overall_status}\n\n"
    )

    f.write(
        "Summary\n"
    )

    f.write("-" * 70 + "\n")

    f.write(
        f"PASS    : {status_counts['PASS']}\n"
    )

    f.write(
        f"PARTIAL : {status_counts['PARTIAL']}\n"
    )

    f.write(
        f"MISSING : {status_counts['MISSING']}\n"
    )

    f.write(
        f"N/A     : {status_counts['N/A']}\n\n"
    )

    for index, item in enumerate(
        results,
        start=1
    ):

        f.write(
            f"{index}. {item['requirement']}\n"
        )

        f.write(
            f"   Status   : {item['status']}\n"
        )

        f.write(
            f"   Evidence : {item['evidence']}\n"
        )

        f.write(
            f"   Details  : {item['details']}\n"
        )

        if item["recommended_action"]:

            f.write(
                f"   Action   : {item['recommended_action']}\n"
            )

        f.write("\n")


# ============================================================
# 30. DISPLAY FINAL AUDIT
# ============================================================

print()
print("=" * 70)
print("AI-POWERED BANKING CUSTOMER SUPPORT")
print("FINAL PROJECT REQUIREMENT AUDIT")
print("=" * 70)

print()
print(f"Project: {BASE_DIR}")

print()
print("-" * 70)

for index, item in enumerate(
    results,
```

```
    start=1
):

    print(
        f"{index:02d}. "
        f"{format_status(item['status']):10s} "
        f"{item['requirement']}"
    )

print()
print("=" * 70)
print("AUDIT SUMMARY")
print("=" * 70)

print(
    f"PASS    : {status_counts['PASS']}"
)

print(
    f"PARTIAL : {status_counts['PARTIAL']}"
)

print(
    f"MISSING : {status_counts['MISSING']}"
)

print(
    f"N/A     : {status_counts['N/A']}"
)

print()
print(
    f"OVERALL STATUS: {overall_status}"
)

print()
print("=" * 70)
print("REPORT FILES")
print("=" * 70)

print(

    AUDIT_OUTPUT
)

print(
    AUDIT_TEXT_OUTPUT
)

print()
print("=" * 70)

if overall_status == "READY":

    print(
        "■ All automatically verifiable mandatory requirements passed."
    )

elif overall_status == "NEEDS MANUAL VERIFICATION":

    print(
        "■ No mandatory requirement is missing, "
        "but some items require manual verification."
    )

else:

    print(
        "■ One or more mandatory requirements need attention."
    )

print("=" * 70)
```

21. Audit Result

Final automated audit result reported by the user: 20 PASS, 1 PARTIAL, 0 MISSING, 1 N/A. Requirements 01–18 and 20–21 passed. Documentation was marked PARTIAL because some documentation elements require manual verification. Deployment was marked N/A because deployment is optional in the project specification.

This is not a missing mandatory project component. The project has no automated audit item marked MISSING.

22. Limitations

• Final DistilBERT accuracy is 88.67%, not 90%+. The official test set was protected from repeated tuning.

• Some semantically similar banking intents remain difficult to distinguish.

• Confidence does not guarantee semantic correctness; safety and escalation controls are therefore necessary.

• The current Generative AI layer is controlled/local rather than a claim of unrestricted external LLM generation.

• Deployment is optional and is not required for the current final local demonstration.

23. Future Enhancements

• Evaluate stronger transformer architectures such as DeBERTa on a separate development split if future compute is available.

• Improve difficult intent pairs with targeted data augmentation and class-aware training on development data.

• Add production-grade authentication, logging, monitoring and human-agent workflow integration.

• Integrate an approved external LLM/RAG knowledge base for richer response generation with strict grounding.

• Deploy the complete Streamlit application to a suitable hosting environment if required.

24. Final Submission Procedure

• Keep the original working project untouched as a backup.

• Submit the FINAL_PROJECT/AI-Powered Banking Customer Support folder or a ZIP of it, according to institute instructions.

• Include app.py, README.md, requirements.txt, dataset artifacts, src scripts, results, notebooks and final model checkpoint as required.

• Do not include .venv unless specifically requested; it is normally recreated from requirements.txt.

• Run streamlit run app.py once before submission and capture screenshots.

• Include screenshots of the Support Assistant, risk/escalation behavior and Model Analytics.

• Include this PDF as the consolidated project documentation if the submission portal accepts PDF documentation.

• For viva, explain the pipeline, baseline comparison, 77-class DistilBERT model, confidence policy, risk layer, GenAI response layer and business value.

25. Final Project Status

Component	Status
Dataset	COMPLETE
NLP preprocessing	COMPLETE
EDA	COMPLETE
ML baseline	COMPLETE
DistilBERT	COMPLETE
Official evaluation	COMPLETE
Confidence	COMPLETE
Escalation	COMPLETE
Risk detection	COMPLETE
Generative AI	COMPLETE
Conversation intelligence	COMPLETE
Streamlit	COMPLETE

Component	Status
Error analysis	COMPLETE
Requirement audit	20 PASS / 1 PARTIAL / 0 MISSING / 1 N/A
Deployment	OPTIONAL / N/A
Overall project	SUBMISSION-READY AFTER FINAL STREAMLIT TEST

Project guide conclusion: The ML development phase is complete. Do not retrain or tune the final model against the official test set. The remaining action is the final Streamlit application test and screenshot evidence, followed by submission packaging.

DOCUMENTATION REQUIREMENTS — FINAL COVERAGE CHECK

Additional sections added to ensure the project documentation explicitly addresses the requested documentation checklist.

Important: The previous PDF already covered most of the requested items. This addendum explicitly adds the three areas that were not sufficiently explicit: Data Dictionary, Prompt Engineering Approach, and Business Impact. It also provides a complete requirement-by-requirement coverage table.

1. Complete Documentation Requirement Coverage

Required item	Coverage in final PDF	Status
Business problem	Problem Statement	✓
Business use cases	Business Purpose and Applications	✓
Dataset	Dataset section	✓
Data dictionary	New section below	✓
Data preprocessing	Data Preparation and EDA	✓
EDA	Data Preparation and EDA	✓
ML baseline	Machine Learning Baseline	✓
Deep Learning methodology	Deep Learning — DistilBERT	✓
GenAI methodology	Generative AI and Conversation Intelligence	✓
Prompt engineering approach	New section below	✓
Confidence and escalation mechanism	Confidence, Risk and Escalation	✓
System architecture	End-to-End System Architecture	✓
Streamlit application	Streamlit Business Application	✓
Evaluation metrics	Official Model Evaluation	✓
Results	Official Model Evaluation / Final Status	✓
Business impact	New section below	✓
Limitations	Limitations	✓
Future enhancements	Future Enhancements	✓

2. Data Dictionary

The official dataset contains two primary fields, text and category. During preprocessing, additional analytical/modeling fields were created in the processed datasets: text_length and label. The project specification identifies text as the customer query and category as the target banking intent. ■filecite■turn2file0■L212-L231■

Field	Type / form	Meaning	Usage
text	Text / string	Customer's banking-support query	Input to NLP/TF-IDF/DistilBERT
category	Categorical string	Official banking intent/category	Target class before numeric encoding
text_length	Numeric integer	Length of cleaned customer query	EDA and text-quality analysis
label	Numeric integer	Numeric ID mapped to official category	DistilBERT classification target

Label mapping: The final mapping contains 77 official intent classes, represented through label2id/id2label mappings and num_classes=77. The official test set contains 40 examples per category. ■filecite■turn0file0■L2-L8■

3. GenAI Methodology

The final implementation uses a controlled, dependency-light response-generation layer rather than claiming unrestricted external LLM generation. The response layer receives the customer query together with the predicted intent, confidence/risk context and escalation decision. It then follows a conservative policy: low confidence → safe escalation/clarification; medium confidence → customer verification; high confidence → category-aware support response subject to semantic safety checks.

This design directly supports the project requirement that the GenAI response use the query, predicted category, confidence and support context while reducing hallucination risk. The project specification also calls for professional, context-aware responses and hallucination controls. ■filecite■turn2file0■L102-L117■

4. Prompt Engineering Approach

Important implementation note: The current final project does not depend on an external LLM API for the response layer, so there is no external provider-specific prompt template to claim as an executed production prompt. Instead, the project uses a controlled response-generation policy.

The prompt-engineering principles documented for future/LLM integration are:

• Role: define the assistant as a banking customer-support assistant.

• Context: provide the customer query, predicted intent, confidence band, risk flags and relevant support context.

• Grounding: restrict the answer to the known intent/support context; do not invent account-specific facts.

• Uncertainty control: when confidence is low or risk flags are present, ask for clarification or escalate instead of guessing.

• Output control: return a concise, professional support response and recommended action.

• Safety: never expose sensitive banking information, invent transactions, or present an uncertain classification as confirmed.

• Escalation: use the risk decision as a hard safety layer around any future LLM response.

These principles are consistent with the project requirement for context-aware GenAI responses and hallucination controls, but they should not be represented as evidence of an external LLM prompt execution in the current implementation. ■filecite■turn2file0■L102-L117■

5. Business Impact

Expected business impact: The system is designed to reduce repetitive manual intent-routing work, prioritize uncertain or high-attention cases, improve consistency of first-level responses, and give support teams a single analytics view of customer issues.

The measurable model result demonstrates that the transformer improves the official-test accuracy over the TF-IDF + Logistic Regression baseline by approximately 0.88 percentage points (87.79% → 88.67%). The system also identifies 20.69% of official-test predictions as low confidence under the current <60% policy, giving a concrete mechanism for directing uncertain cases toward human support rather than blindly automating them.

Business impact should be described as expected/operational impact, not as a measured monetary saving, because no production workload, response-time, customer-satisfaction or cost dataset was provided in the project evidence. This distinction keeps the documentation evidence-based.

6. Recommended Viva Explanation

A simple explanation is: “The project classifies 77 banking customer-support intents using a fine-tuned DistilBERT model. A TF-IDF + Logistic Regression model provides the traditional baseline. The transformer reaches 88.67% accuracy on the reserved official test set. Confidence and risk detection determine whether the system can respond automatically, should verify the customer's intent, or should escalate to a human. A controlled GenAI response layer and Streamlit interface turn the classifier into a practical support application.”

Final conclusion: With this addendum attached to the original documentation, all requested documentation headings are explicitly covered in the single combined PDF.